CASHFLOW COPILOT - REMAINING WORK CHECKLIST

Last updated	`2026-11-07`
Repo	`https://github.com/sidg2hp/paypal`
Live commit	`b8ad4ae (master, clean and pushed)`
Deadline	`2026-11-12, 12:00 pm PT = 2026-11-13, 01:30 IST`
Winners	`2026-12-21`

> **This file is intentionally LOCAL ONLY.** It is listed in `.gitignore` and must not be committed. The committed, judge-facing versions of this material live in:
>
> `docs/SUBMISSION.md` Devpost fields, checklist, sponsor selection
> `docs/DEMO_SCRIPT.md` Video shot list and voiceover
> `docs/SANDBOX_SETUP.md` PayPal sandbox setup
> `docs/DEPLOY.md` Render and Docker
> `docs/FINAL_HANDOFF.md` Owner-only steps and final commands
> `README.md` Pitch, live-vs-demo table, quick start

STATUS

12/16 tasks complete. The 4 remaining tasks need YOUR accounts, credentials, public hosting, or Docker:

- [ ] 1. Real PayPal Sandbox credentials + committed evidence
- [ ] 2. Hosted demo on Render
- [ ] 3. Demo video + Devpost submission
- [ ] 4. Docker build verification

Suggested order (5 days):
Day 1 Part 1: PayPal sandbox + evidence
Day 2 Part 2: Render deploy + webhook
Day 3 Part 3: record and upload video
Day 4 Part 5: Devpost submission
Day 5 Buffer / re-record if needed

PART 0 - ONE-TIME PREREQUISITES

1. Node.js 22 or newer installed. Check:

   ```
   node --version
   ```

   (The repo was verified on Node 24.21.0.)
2. Git installed and authenticated to GitHub:

   ```
   git --version
   git remote -v
   ```

   Expected: `https://github.com/sidg2hp/paypal.git`
3. Clone the repo somewhere clean:

   ```
   git clone https://github.com/sidg2hp/paypal.git
   cd paypal
   npm ci
   npm test
   ```

   Expected: 58 tests passing.
4. Start the app once in demo mode to confirm the baseline:

   ```
   npm start
   ```

   Open http://localhost:3000
   You should see "MOCK MODE" and four seeded invoices.
5. Create a folder for personal notes/secrets OUTSIDE the repo, or keep a `.env` file locally. NEVER commit `.env`, client secrets, or access tokens.

PART 1 - PAYPAL SANDBOX: CLIENT ID, SECRET, WEBHOOK, EVIDENCE

Goal: switch the app from mock mode to real PayPal Sandbox, prove invoice create/send/read/remind, commit a real evidence JSON, and verify a webhook.

1.1 Create a PayPal Developer account

a. Open https://developer.paypal.com/dashboard/
b. Sign in with your normal PayPal account (or create one).
c. Complete any developer profile prompts.

1.2 Create a Sandbox REST app and copy the credentials

a. Open https://developer.paypal.com/dashboard/applications/sandbox (Dashboard -> Apps & Credentials -> Sandbox tab)
b. Click "Create App" (or "Create App" under REST API apps).
c. App name: cashflow-copilot
d. App type: Merchant (default)
e. Click "Create App".
f. You now see:
Client ID: a long string
Secret: click "Show" or the eye icon

Copy BOTH into a local password manager or your local `.env` file. These are SANDBOX credentials. Do not use live credentials.

1.3 Create a Sandbox personal (buyer) account

a. Open https://developer.paypal.com/dashboard/accounts
b. Make sure the "Sandbox" tab is selected.
c. Click "Create Account".
d. Account type: Personal
e. Country: your country
f. Email: keep the generated one, or set your own (e.g. sb-buyer@business.example.com)
g. Password: generate or set one, and save it.
h. Click "Create Account".
i. Click the new account -> "View/Edit account" to see the email and password.

This buyer account is what you will use to pay the invoice during the demo.

1.4 Run the app in Sandbox mode locally

PowerShell:

```
$env:PAYPAL_CLIENT_ID="<your-client-id>"
$env:PAYPAL_CLIENT_SECRET="<your-secret>"
$env:AI_PROVIDER="deterministic"   # keeps the demo fast; optional
npm start
```

Or create a local `.env` (not committed) and use a loader, or pass env inline.

Verify:

```
Invoke-RestMethod http://localhost:3000/api/health
```

Expected:

```
status       : ok
adapter      : paypal-sandbox
mode         : sandbox
capabilities.checkout : True
```

In the browser, the sidebar and banner should say SANDBOX MODE.
If it still says mock: one credential is missing, or the server was started before the environment variables were set. Stop and restart.

1.5 Click through a real Sandbox invoice

a. Open http://localhost:3000
b. Click "Draft an invoice from a sentence".
c. Use this exact prompt:
`Invoice Acme Design Co <accounts@acme.example> for 2 brand review sessions at $125 each, tax 8%, due in 15 days`
d. Review the modal. Customer, 2 sessions at $125, 8% tax, total $270.00.
e. Replace the email with your sandbox buyer email if you prefer.
f. Click "Approve & send with PayPal".
g. Open the invoice drawer.
   - PayPal invoice id should be present.
   - Timeline shows "Invoice sent".
h. Click "Send gentle reminder" and confirm.
i. Optional: open https://www.sandbox.paypal.com and log in as the buyer to see the invoice in the buyer account.

If the send fails, see 1.10.

1.6 Generate committed evidence

Run:

```
$env:PAYPAL_CLIENT_ID="<your-client-id>"
$env:PAYPAL_CLIENT_SECRET="<your-secret>"
$env:SANDBOX_TEST_EMAIL="<your-sandbox-buyer-email>"
npm run sandbox:check
```

The script:

- creates a $12.34 sandbox invoice through the app
- sends it through PayPal Invoicing v2
- reads it back with GET /v2/invoicing/invoices/{id}
- sends a reminder
- verifies the local hash chain
- writes docs/sandbox-evidence/<timestamp>.json

Expected output ends with:

```
Sandbox evidence: success
... PayPal-Debug-Id <id>
wrote docs/sandbox-evidence/<timestamp>.json
```

The JSON contains no client id, secret, token, or headers. Open it and sanity check:

- outcome: "success"
- remote.total: "12.34"
- remote.status: something like SENT
- steps[].debugId present

1.7 Commit the evidence

```
git add docs/sandbox-evidence/*.json
git commit -m "docs: add real PayPal sandbox evidence"
git push origin master
```

Do not edit a failed evidence file to make it look successful. Fix the cause and generate a new run. A failing file with a debug id is still useful evidence of debugging, but it does not prove a successful integration.

1.8 Create and verify a real webhook

PayPal cannot call localhost. Do this AFTER Part 2 (Render), or use a tunnel.

a. Open the Render URL in a browser, e.g.
`https://cashflow-copilot.onrender.com`

b. In the PayPal Developer Dashboard:
Apps & Credentials -> your app -> Webhooks -> Add Webhook
(If you do not see it there, open the "Webhooks" page in the dashboard.)

c. Webhook URL:
`https://<your-service>.onrender.com/api/webhooks/paypal`

d. Event types: tick these
- INVOICING.INVOICE.SENT
- INVOICING.INVOICE.PAID
- INVOICING.INVOICE.CANCELLED
- INVOICING.INVOICE.VOIDED
- INVOICING.INVOICE.REFUNDED
- PAYMENT.CAPTURE.COMPLETED

e. Click Save.

f. Copy the Webhook ID (looks like WH-XXXXXXXXXX).

g. In Render -> your service -> Environment:

```
PAYPAL_WEBHOOK_ID = <the webhook id>
PAYPAL_CLIENT_ID  = <client id>
PAYPAL_CLIENT_SECRET = <secret>
PUBLIC_BASE_URL = https://<your-service>.onrender.com
TRUST_PROXY = 1
```

Save and let it redeploy.

h. Trigger a payment:
- Open the hosted app, create/send an invoice.
- Open the pay page from the invoice drawer -> "Create payment link".
- Pay with the sandbox buyer.
- Return to the app. The invoice should move to PAID.

i. Open the invoice drawer. The timeline should show:
"Payment received" with "Verified PayPal webhook" and an event id.
The state change must happen through the verified webhook, not only the capture call.

j. Negative test: in a terminal, POST to the webhook with no PayPal headers.
Use curl.exe (Windows ships it) in one line:

```
curl.exe -i -X POST https://<your-service>.onrender.com/api/webhooks/paypal -H
"Content-Type: application/json" -d "
{\"id\":\"evt_fake\",\"event_type\":\"INVOICING.INVOICE.PAID\",\"resource\":
{\"invoice_id\":\"inv_seed_001\"}}"
```

Expected: HTTP 401 and the invoice does NOT change.

1.9 Test Checkout (customer side)

a. In the app, open a SENT invoice drawer.
b. Click "Create payment link".
c. Click "Open checkout".
d. In sandbox mode, the button redirects to PayPal. Log in with the sandbox buyer and approve.
e. On return, the app calls capture and marks the invoice PAID.
f. Confirm the timeline shows PAYMENT.CAPTURE.COMPLETED with source paypal-checkout.

1.10 Troubleshooting

Symptom: still "MOCK MODE"
- One of PAYPAL_CLIENT_ID / PAYPAL_CLIENT_SECRET is missing.
- The env vars were set in a different terminal than the running server.
- Restart `npm start`.

Symptom: "PayPal authentication failed"
- Wrong id/secret, or a LIVE app used against the sandbox base URL.
- Regenerate the secret and retry.

Symptom: "PayPal request failed (422)"
- Invalid invoice payload, usually the recipient email or item amount.
- Check the email is a valid sandbox buyer address.

Symptom: "PayPal request failed (404)"
- The invoice id belongs to a different PayPal environment.

Symptom: webhook returns 401
- Missing PAYPAL-* headers (only PayPal can generate them).
- Wrong PAYPAL_WEBHOOK_ID.
- Signature verification failed; check the server logs for the error code.

Symptom: webhook accepted but no state change
- Duplicate event id, or the event is stale for the current lifecycle status.

Symptom: checkout capture fails
- The buyer never approved the order, or the order expired.
- Recreate the payment link and try again.

Hard rule: never put real LIVE PayPal credentials in this repo or a public demo. Use sandbox only.

PART 2 - HOSTED DEMO ON RENDER

Goal: a public HTTPS URL judges can open, with the sandbox adapter and a working webhook.

2.1 Push the repo

Already done. Confirm:

```
git status
git log --oneline -1
git push origin master
```

2.2 Create a Render account

a. Open https://dashboard.render.com/
b. Sign up with GitHub (easiest; it lets Render read the repo).
c. Authorize Render to access the repository.

2.3 Deploy with the Blueprint

a. In Render, click "New +" -> "Blueprint".
b. Select the repository sidg2hp/paypal.
c. Render reads render.yaml and shows the service "cashflow-copilot".
d. Click "Apply".
e. When prompted for `sync: false` values, fill:

```
PAYPAL_CLIENT_ID      = <sandbox client id>
PAYPAL_CLIENT_SECRET  = <sandbox secret>
PAYPAL_WEBHOOK_ID     = <webhook id>  (can leave blank on first deploy)
PUBLIC_BASE_URL       = https://<service-name>.onrender.com
OPENAI_API_KEY        = optional; leave blank for deterministic planner
```

f. Render generates DEMO_RESET_SECRET automatically because render.yaml uses generateValue: true.
g. Wait for the deploy to finish (2-4 minutes).

2.4 Verify the deployment

a. Open `https://<service-name>.onrender.com`

b. The dashboard should load with four seeded invoices.

c. Open: `https://<service-name>.onrender.com/api/health`
Expected:

```
"status": "ok"
"mode": "sandbox"      (if PayPal env vars are set)
"capabilities": { "checkout": true, "paypalWebhooks": true, ... }
```

d. If mode is "mock", the credentials did not reach the service. Fix the env vars in Render -> Environment and redeploy.

2.5 Warm the demo before recording

Render free instances spin down after about 15 minutes idle and take about a minute to wake. Before recording or sharing: Open `https://<service-name>.onrender.com/api/health` in a browser and wait for a 200 response. Do not record the cold start.

2.6 Security settings for a public demo

- Leave PAYPAL_* on sandbox credentials only.
- DEMO_RESET_SECRET is already generated; keep it. If you want to reset the demo from the UI, the button works from the same browser (same origin). To reset from curl, send: `Authorization: Bearer <DEMO_RESET_SECRET>`
- Do not put AI keys on a public demo if you care about spend. The deterministic planner is enough for the demo and is labelled honestly.
- Rate limits are on by default (240 reads/min, 60 writes/min per IP).
- Same-origin checks block cross-site writes.

2.7 Persistence (optional)

Free Render instances have an ephemeral filesystem. The demo reseeds on restart, which is fine for judges. For durable state:

- Upgrade to a paid instance.
- Uncomment the disk block in render.yaml (mountPath /var/data, sizeGB 1).
- Add DATA_FILE=/var/data/store.json to env vars.
- Redeploy.

2.8 Custom domain (optional)

Render -> Settings -> Custom Domains. If you use one, update PUBLIC_BASE_URL to the custom HTTPS origin and update the PayPal webhook URL to match.

2.9 Troubleshooting

- Build fails: check the Render logs. `npm ci` needs package.json and package-lock.json, both committed.
- App starts but 502s: check the start command is `node src/server.js` and the PORT env is set by Render automatically.
- Health check fails: `/api/health` must return 200; check the service logs.
- Webhook 404: the path must be exactly `/api/webhooks/paypal`.
- Webhook 401: PAYPAL_WEBHOOK_ID missing or wrong, or the cert URL was rejected.
- Images/AG Grid missing: the vendor files under public/vendor are committed; confirm they are in the deployed commit.

PART 3 - DEMO VIDEO (UNDER 3 MINUTES)

Goal: a public YouTube video that proves the product works end-to-end and makes the novelty obvious in the first 15 seconds.

3.1 Pre-flight checklist

- [ ] Hosted URL is warm and in sandbox mode (check /api/health).
- [ ] PayPal Developer Dashboard is open in a second tab, showing the sandbox invoice created by the app.
- [ ] Sandbox buyer account is logged in or credentials are ready.
- [ ] Demo is reset so the queue shows the seeded story.

- [ ] One invoice is ready to pay for the cold open.
- [ ] Browser: clean profile, bookmarks hidden, notifications off, zoom ~110%.
- [ ] Screen resolution 1920x1080. Record at 1080p, 30 fps.
- [ ] Mic tested. Record in a quiet room.
- [ ] No copyrighted music. No third-party logos beyond PayPal's own product UI.

3.2 Recording setup

Recommended: OBS Studio (free).
- Scene: Window Capture (Chrome) or Display Capture.
- Output: 1920x1080, 30 fps, MP4.
- Audio: microphone only, no desktop audio unless you need click sounds.
- Test a 15-second clip and watch it back before the real take.
Alternatives: Windows Game Bar (Win+G), or QuickTime on macOS.

3.3 The script (target 2:40)

0:00-0:15 COLD OPEN - THE MONEY MOMENT
On screen: the hosted pay page, clicking "Pay with PayPal", approving in the sandbox, and the invoice flipping to PAID. Cut to the forecast bar moving.
Voiceover:
"Freelancers lose cash to invoices that are already late. This is the AI operator that tells you which one to chase - and only moves money after you approve. It runs on PayPal."

0:15-0:35 WHO IT IS FOR
On screen: the command center. Point at outstanding, overdue, 30-day forecast.
Voiceover:
"This is a small design studio with open invoices. The command center ranks what needs attention, explains why, and shows what is actually expected to arrive."

0:35-1:10 INTENT TO A REVIEWABLE PAYPAL ACTION
On screen: "Draft an invoice from a sentence", then the review modal. Change one value, point at the recalculated total and currency.
Type:
`Invoice Acme Design Co <accounts@acme.example> for 2 brand review sessions at $125 each, tax 8%, due in 15 days`
Voiceover:
"A sentence becomes a validated draft. The model extracted the customer, the line items, the tax, and the due date. Every total is recalculated server-side in cents - the model never does the arithmetic. I can edit anything before it becomes a PayPal invoice."

1:10-1:35 AGENT PLAN WITH AN APPROVAL GATE
On screen: ask "Who should I chase today?" Show the plan card with two steps and checkboxes. Untick one, click "Approve selected". Show the toast and the new timeline event.
Voiceover:
"The agent can propose five actions and nothing else. It cannot invent an invoice id, and it cannot execute. I approve exactly the steps I want. The policy engine decides whether a reminder is even allowed, and shows the rule."

1:35-2:05 PAYPAL DEPTH
On screen: PayPal Developer Dashboard tab showing the sandbox invoice and amount. Back to the app: drawer, PayPal invoice id, create payment link, hosted pay page, PayPal approval, PAID.
Voiceover:
"The invoice is real in PayPal Sandbox. Tax is allocated per item so PayPal's total matches the app. The customer gets a hosted PayPal checkout. The invoice only becomes paid after PayPal confirms the capture - not when the button is clicked."

2:05-2:30 THE DEPTH BEAT - EVIDENCE, NOT ADJECTIVES
On screen: quick cuts
1. `npm run backtest` output: 2.36 d vs 7.98 d, 70.4% better.
2. "Verify audit chain" showing the chain valid.
3. A policy rule such as REMINDER_LIMIT_REACHED in the drawer.
Voiceover:
"The forecast is not a chart. It is due date plus each payer's median lateness - 70 percent lower error than a due-date baseline in the committed backtest. Every timeline event is hash-chained, and every recommendation carries the policy rule that allowed it."

2:30-2:45 IMPACT AND CLOSE
On screen: the queue, then an end card with repo URL and hosted demo URL.
Voiceover:
"Cashflow Copilot turns a sentence into a reviewable PayPal action, then turns verified payment events into a calm, explainable collection plan. Open source, runs offline, and the PayPal sandbox path is two environment variables away. Repo and demo are in the description."

3.4 Features and novelty talking points

Use these in the video description and Devpost write-up.

One-sentence pitch:
"The AI accounts-receivable operator that predicts when each invoice will actually be paid, proposes the one action that moves cash this week, and only moves money through PayPal after you approve."

Eight features:

1. Natural language -> validated invoice proposal; every total recalculated server-side in integer cents.
2. Agent plan with a per-step approval gate; five allowlisted financial tools; 15-minute plan TTL.
3. Payer-aware payment-date forecast, not just a due-date chart.
4. Collections policy as data, with named rules and numeric inputs attached to every recommendation.
5. PayPal Invoicing v2 (create/send/read/remind/cancel) and Orders v2 checkout.
6. Verified PayPal webhooks: signature verification, cert-host allowlist, duplicate and stale-event protection.
7. SHA-256 hash-chained audit trail, verifiable from the UI or /api/audit.
8. Two-sided product: merchant command center plus a hosted customer pay page.

Novelty paragraph:
"Most accounts-receivable agents stop at generating an invoice. Cashflow Copilot owns the harder half: it predicts when each invoice will actually be paid using the payer's own history, makes the collections policy explicit and inspectable, keeps the AI inside a five-tool allowlist, and records every outcome in a hash-chained audit trail. PayPal Invoicing handles the merchant side, PayPal Checkout handles the customer side, and verified webhooks close the loop."

Proof numbers:

- 58 offline tests
- 16/16 golden parser/planner cases
- 70.4% lower MAE than a due-date baseline (2.36 d vs 7.98 d)
- 0 OpenAPI validation errors (Redocly; 18 style warnings)
- 4 seeded invoices, 4 PayPal surfaces (Invoicing, Checkout, Webhooks, OAuth)

3.5 Recording tips

- Record the whole run in one take if possible; cut only dead air.
- Keep the mode banner visible. Do not crop out "SANDBOX MODE" or "MOCK MODE".
- Move the mouse deliberately; do not hunt for buttons.
- If you make a mistake, pause 2 seconds and restart the section; edit later.
- Do not show terminal `npm install` or setup screens.
- Do not show personal tabs, email, or notifications.
- Speak slightly slower than feels natural; judges may watch at 1x.

3.6 What not to claim

- Do not say "fully autonomous". The merchant approves every step.
- Do not say "verified webhook" for an event created by /api/webhooks/simulate.
- Do not say "production ready". The default store is in memory.
- Do not say "AP2 compliant". The trust loop is aligned with AP2; AP2 is not implemented.
- Do not use copyrighted music or unlicensed marks.

3.7 YouTube upload

a. Open https://www.youtube.com/upload
b. Title: Cashflow Copilot - AI accounts receivable on PayPal
c. Visibility: Public (not Unlisted).
d. Not made for kids.
e. Description: one-paragraph summary, repo URL, hosted demo URL, and one line: "Demo mode is labelled in the UI; the PayPal sandbox path is shown in this video." Adjust to match what you actually recorded.
f. Upload at least 24 hours before the deadline. Processing can take hours.
g. Paste the URL into Devpost and into docs/SUBMISSION.md if you keep a copy.

PART 4 - DOCKER (OPTIONAL BUT REQUESTED)

Goal: prove the app builds and runs as a container. Docker is not installed on the machine used to build this repo, so this is the one unverified path.

4.1 Install Docker Desktop on Windows

a. Open https://www.docker.com/products/docker-desktop/
b. Download Docker Desktop for Windows.
c. Run the installer.
d. When prompted, enable WSL 2 (Windows Subsystem for Linux). If WSL is missing:

```
wsl --install
```

then reboot.
e. Start Docker Desktop and wait until it says "Engine running".
f. Verify in PowerShell:

```
docker --version
docker run --rm hello-world
```

4.2 Build the image

From the repo root:

```
docker build -t cashflow-copilot .
```

Expected: the build finishes successfully. The image:

- uses node:22-bookworm-slim
- runs as the non-root `node` user
- copies src, public, policies, openapi, scripts, tests, docs, postman
- includes a HEALTHCHECK against /api/health
- installs no runtime dependencies

4.3 Run the mock demo

```
docker run --rm -p 3000:3000 cashflow-copilot
```

Open http://localhost:3000
Check:

```
Invoke-RestMethod http://localhost:3000/api/health
```

Expected: mode "mock", status "ok".
Check the container health:

```
docker ps
```

The STATUS column should eventually show "(healthy)".

4.4 Run with PayPal Sandbox credentials

Option A - explicit env vars (one line; works in PowerShell and cmd):

```
docker run --rm -p 3000:3000 -e PAYPAL_CLIENT_ID="<client-id>" -e
PAYPAL_CLIENT_SECRET="<secret>" -e PAYPAL_WEBHOOK_ID="<webhook-id>" -e
PUBLIC_BASE_URL="http://localhost:3000" cashflow-copilot
```

Option B - local env file (never commit it):

```
docker run --rm -p 3000:3000 --env-file .env cashflow-copilot
```

Check `/api/health` again; mode should be "sandbox".

4.5 Persistence inside Docker (optional)

PowerShell:

```
New-Item -ItemType Directory -Force data
docker run --rm -p 3000:3000 -v "${PWD}\data:/data" -e DATA_FILE=/data/store.json
cashflow-copilot
```

cmd.exe:

```
mkdir data
docker run --rm -p 3000:3000 -v "%cd%\data:/data" -e DATA_FILE=/data/store.json
cashflow-copilot
```

The JSON store is written atomically inside the mounted folder.

4.6 Common Docker problems

- "docker: command not found": Docker Desktop is not installed or PATH is stale. Restart the terminal after installing.
- "error during connect ... docker daemon is not running": start Docker Desktop.
- "port is already allocated": another process uses 3000. Use -p 3001:3000 and open http://localhost:3001.
- Build is slow the first time: the node base image is large. Later builds are cached.
- Health check shows unhealthy: the container needs a few seconds; check `docker logs <container-id>`.
- Windows line endings in shell scripts are not an issue here (no shell entry point).

4.7 Cleanup

```
docker ps
docker stop <container-id>
docker image rm cashflow-copilot
docker system prune -f   # optional; removes unused images/containers
```

PART 5 - DEVPOST SUBMISSION

5.1 Open the form

a. Open https://paypalaihackathon.devpost.com/

b. Click "Submit project" / "Join hackathon" as appropriate.

c. If you have teammates, invite them and make sure they ACCEPT before the deadline.

5.2 Fill the fields

Project name:
Cashflow Copilot

Elevator pitch:
"The AI accounts-receivable operator for PayPal. It predicts when each invoice will actually be paid, proposes the one action that moves cash this week, and executes it through PayPal only after you approve. Every step is explainable, policy-gated, and hash-chained."

About the project:
Copy the full markdown from docs/SUBMISSION.md -> "About the project".

Built with:
PayPal Invoicing v2, PayPal Orders v2 (Checkout), PayPal Webhooks v1, Node.js 22, vanilla JavaScript, AG Grid Community, APIMatic (OpenAPI 3.1 + PayPal Server SDK Context Plugin), Render, Postman, OpenAI-compatible LLMs (OpenAI / xAI / Gemini / local Ollama), SHA-256.

Try it out:
https://<your-service>.onrender.com

Video:
<your public YouTube URL>

Repository:
https://github.com/sidg2hp/paypal

Tools / AI explanation:
Copy from docs/SUBMISSION.md -> "Tools used (AI)".

5.3 Select sponsor prizes

Tick the sponsor tools you used. At minimum:

☐ AG Grid

- [ ] APIMatic
- [ ] Render
- [ ] Postman (if listed)

Prize stacking allows one Grand Prize or Honorable Mention plus one sponsor prize. A repository mention alone does not enter you for a sponsor prize.

5.4 Upload gallery images

Use these committed files, in this order:

1. docs/assets/dashboard.png
2. docs/assets/copilot-plan.png
3. docs/assets/pay-page.png
4. docs/assets/audit-drawer.png

Keep the mode banner visible in every screenshot.

5.5 Final checklist (Stage One pass/fail)

- [ ] Public GitHub repo with all source and instructions.
- [ ] MIT license visible in the repo About sidebar.
- [ ] Hosted demo URL works in a clean browser.
- [ ] Video is public on YouTube and under 3:00.
- [ ] Video shows the project working end-to-end.
- [ ] Text description explains features and functionality.
- [ ] Tools section identifies the AI models/services and how they are used.
- [ ] Meaningful use of PayPal and AI.
- [ ] No third-party trademarks or copyrighted music without permission.
- [ ] Teammates invited AND accepted.
- [ ] Sponsor tools selected on the form.
- [ ] Demo, credentials, and video remain live until 2026-12-21.

- [ ] Repo and video frozen after submission until winners are announced.

5.6 After you submit

- Do not edit the repo, video, or linked materials until after 2026-12-21.
- Keep the Render service alive. If it is on the free tier, it may spin down; that is fine for judges as long as it wakes, but a warm demo is better.
- Keep the sandbox credentials valid.
- Save a screenshot of the Devpost confirmation page.

PART 6 - COMMAND CHEAT SHEET

Start demo:

```
npm start
```

Run all checks:

```
npm run verify
```

Tests only:

```
npm test
```

Golden evals:

```
npm run eval
```

Backtest:

```
npm run backtest
```

Real sandbox evidence:

```
$env:PAYPAL_CLIENT_ID="..."
$env:PAYPAL_CLIENT_SECRET="..."
$env:SANDBOX_TEST_EMAIL="..."
npm run sandbox:check
```

Health:

```
Invoke-RestMethod http://localhost:3000/api/health
```

Reset demo (local, no secret):

```
Invoke-RestMethod -Method Post http://localhost:3000/api/demo/reset -ContentType
"application/json" -Body "{}"
```

Reset demo (protected):

```
Invoke-RestMethod -Method Post http://localhost:3000/api/demo/reset `
  -Headers @{ Authorization = "Bearer $env:DEMO_RESET_SECRET" } `
  -ContentType "application/json" -Body "{}"
```

Docker:

```
docker build -t cashflow-copilot .
docker run --rm -p 3000:3000 cashflow-copilot
```

Generate a reset secret:

```
node -e "console.log(require('crypto').randomBytes(24).toString('hex'))"
```

PART 7 - FEATURES AND NOVELTY (PASTE-READY)

Short version:
Cashflow Copilot is a human-controlled AI accounts-receivable operator for PayPal. It turns a sentence into a validated invoice, predicts when each invoice will actually be paid from payer history, ranks the next best action under an explicit collections policy, and executes only the actions the merchant approves. PayPal Invoicing handles the merchant side, PayPal Checkout handles the customer side, and verified webhooks plus a SHA-256 hash chain close the loop.

What is genuinely different:

1. A measured forecast, not a chart. Payer-aware payment-date prediction with a committed backtest: 70.4% lower MAE than a due-date baseline.
2. A policy engine outside the model. Reminder limits, cooldowns, and send caps are data; every recommendation carries the rule id and inputs.
3. A real agent boundary. Five allowlisted tools, real invoice-id validation, a 15-minute plan TTL, and per-step approval.
4. A verifiable audit trail. Every event is SHA-256 chained; /api/audit recomputes it and reports the first broken link.
5. Two-sided PayPal. Invoicing v2 for the merchant, Orders v2 for the customer, verified webhooks for the outcome.
6. Honest mode labels. The UI and README never present a simulated event as a live PayPal event.

What it is not:

- Not a generic invoice writer.
- Not an autonomous payer.
- Not production-ready multi-tenant software (in-memory store, per-process rate limits, synthetic demo data).

FINAL 10-MINUTE CHECK BEFORE SUBMITTING

1. Open the hosted URL in a private window. Does it load with four invoices and a visible mode banner?
2. Open /api/health. Is status ok and mode correct?
3. Open the YouTube link in a private window. Is it public and under 3:00?
4. Open the GitHub repo in a private window. Is the MIT license visible in the About sidebar?
5. Open docs/SUBMISSION.md and confirm every field is filled.
6. Confirm the Devpost form has the repo, video, hosted URL, and sponsor tools.
7. Click Submit. Save the confirmation screenshot.

Good luck.